Threaded Image Processing

Optimizing Conversion and Resizing Operations

Nicole Balay, Ashanti Boone, Kiishi Ogunbiyi, Hanaa Salim*

Computer Science Department, Bowie State University

Jericho Park, Bowie, Maryland 20715

[1]balayn0411@students.bowiestate.edu

[2]boonea1212@students.bowiestate.edu

[3]ogunbiyii0724@students.bowiestate.edu

[4]salimh0807@students.bowiestate.edu

Abstract– This paper explores the usage of multithreading to decrease image processing time. This proposal analyzes a multithreaded image conversion and resizing approach, focusing on resource utilization and thread synchronization. The experiment results will show a significant reduction in time, which will be used to highlight the effectiveness of using multithreads to process images. This paper also examines the CPU utilization across different Windows and MacOS systems when completing these image processing tasks.

Keywords– Image processing, multithreading, conversion, and resizing.

I. INTRODUCTION

When processing videos and images, efficiency is an essential factor. Traditional single-threaded processing methods fall short of meeting the demands for speed and efficiency, especially with large datasets. Multithreading, which gives a process multiple threads of control, can perform multiple tasks simultaneously. This research will investigate multithreading's efficiency in decreasing image processing time, particularly with large datasets. This will be completed by comparing the processing time of a multithreaded approach with a non-threaded version. By simultaneously executing programs, multithreading significantly reduces processing time and enhances efficiency. This research will also use the image processing tasks to examine potential differences in memory management across different operating systems (like Windows vs MacOS).

To address the challenges posed by single-threading limitations, the following goals and objectives are proposed to leverage multithreading for enhanced image processing efficiency:

Goals:

1. Decrease Processing Time
2. Increase Efficiency
3. Reliability
4. Scalability

Objectives:

1. Design and Implement a Multithreaded Processing Framework
2. Optimize Thread Management
3. Performance Analysis
4. Error Handling and Reliability

This project aims to improve the efficiency and speed of image processing by achieving these goals.

II. LITERATURE REVIEW

The paper Low Latency Image Processing of Transportation System Using Parallel Processing Co-incident Multithreading (PPcM) [1] by Shammi and Sultana et al. covers reducing latency in image processing for transportation systems through multi-threading techniques and parallel processing. It also highlights using a multi-threading approach to handle real-time video and image processing, reducing the time complexity and increasing the frames per second rate significantly compared to traditional methods.

Real-Time Digital Image Processing Using Point Operations in Multithreaded Systems [2] by Samyan and Sahar et al. is a paper that focuses on real-time digital image processing using multi-threading for reliable and efficient output. There is a differentiation between single pixel operations and neighborhood pixel values image processing tasks, and the partition of images to

optimize multi-threading performance without data loss is covered.

Applying GPU and POSIX Thread Technologies in Massive Remote Sensing Image Data Processing [3] by Liu and Chen et al. proposes a dual-parallel processing mechanism based on GPU and POSIX threading technologies to handle massive remote sensing image data. A system where image data is divided into blocks and tiles to manage memory limitations effectively while utilizing GPU for accelerated processing is introduced. This method enhances processing efficiency by enabling simultaneous data I/O and image processing.

Parallel Image Stitching Based on Multithreaded Processing on GPU [4] by Nakov and Mihaylova et al. mentions a technique for parallel image stitching using GPU-based multithreading processes. The approach leverages the computational power of GPUs in conjunction with multi-threading to enhance the speed and efficiency of Stitching large sets of images into a single panoramic image.

Parallel image segmentation using multi-threading and k-means algorithm [5] by Bose and Mukherjee et al. discuss a method for image segmentation that uses multi-threading and k-mean clusters to enhance processing efficiency. The method proposed by the paper reduces segmentation time compared to traditional approaches by dividing an image into multiple parts and assigning each to a unique thread. Various tests with different thread configurations show that parallel processing accelerates image analysis.

The article Image Processing, written by L. S. G. KOVASZNAYt and H. M. JOSEPH, describes two critical operators for image processing, contour enhancement and contour outlining [6]. This experiment was completed using a block diagram to conduct experiments based on light and transparency to determine how it affects an image's contour or outline. They determined that the contour would change depending on positive or negative openness. If identical results occur, then the system is properly aligned. In their experiment, they completed simple image processing; they would like to explore the processing of warping, changing position, and rotating images. These improvements could be applied to X-ray technology.

The article Composable Multi-threading for Python Libraries by Anton Malakhov [7] describes the use of Intel Threading Building Blocks for multicore parallelism as the compatible layer for Python. The experiment was conducted via a case study where they mimicked a recommendation system that generates suggestions for the next movie. Outmost parallelism was used on the application level to process user requests. He learned that the QR decomposition example is 46% faster than the baseline implementation. The case study improved it by 59%. Intel TBB replaced Numba as it has a three times speed-up time. Limitations and future work include how Intel TBB does not work for blocking I/O operations, Intel TBB is only in an experimental stage, and Intel TBBB can only thread in single processes.

The article Performance and Productivity of Parallel Python Programming — A Study with a CFD Test Case by Achim Basermann, Melven Röhrig-Zöllner, and Joachim Illmer [8] discusses C, Fortan, and Python performances. The authors create an environment for performance tests and benchmarks for optimal memory bandwidth performance. The SP peak performance was 1030 GFLOPS, and the DP Performance was 515 GFLOPS. They then compare Python and Forkan performance on single-core, full multi-core CPU, and GPU of test hardware. The authors concluded that Python, without additional libraries, has less computing performance than C and Fortran. They hope communities behind libraries like Numba and NumPy will continue developing their solutions to make parallel programming more efficient on parallel hardware.

The article Image-to-Image Translation: Methods and Applications by Yingxue Pang, Jianxin Lin, Tao Qin, and Zhibo Chen [9]. They provided an overview of I2I works, analyzed essential techniques, discussed the main progress in the community, discussed the effect of I2I on research, and pointed out challenges in the field. The authors decided it was difficult to name one algorithm superior to the others. However, they noticed that supervised methods produce better translated results than unsupervised methods on similar network structures. They know I2I can be applied to tasks like transfer learning for reinforcement learning, image registration, and domain adaption. They hope to see their method applied to these tasks.

The article Optimized Scale-and-Stretch for Image Resizing by Yu-Shuen Wang, Chiew-Lan Tai, Olga Sorkine, Tong-Yee Lee [10]. The authors introduce a scaling and stretching method that allows the resizing of images while keeping prominent features. They used a calculation with vertices and edges to resize images, such as grid line bending

and boundary conditions. In conclusion, they have developed an efficient method for nonlinear optimization to warp images, allowing interactive image resizing. Some limitations include removing quad content when their algorithm comes into contact with it, stretching homogenous quads, and failing to preserve the shapes of prominent lines of arbitrary orientations.

Jin Bo's article, "Improved Parallel Downloading Algorithm for Large File Distribution," [11] introduces a new parallel downloading algorithm that improves efficiency by considering server bandwidth and availability when selecting mirror servers. This adaptive method is tested in a simulation in which an 80 MB file is downloaded from four servers with varying bandwidths (two at 4 Mbps and two at 3 Mbps), controlled by a program that manages server availability. The test results show that this new method reduces download times despite an intermediate router's 14 Mbps bandwidth limit.

Yu Zhang's article, "The studies and implementation for conversion of image file format" [12], studies the compression algorithm by comparing different color models between the two file formats before designing the conversion flow and finally implementing the conversion from BMP to GIF format. The study examined the file structure formats, focusing on each color model. This was necessary because the development environment primarily supports BMPs. The study investigated and compared two different algorithms that could be used during the conversion process. The conversion process was tested and found to produce satisfactory image quality results. The study highlights the varying efficiency and quality among different image file format conversion algorithms While noting the ongoing improvements in the field due to rapid advancements, including updates to GIF formats such as the 87a and 89a editions.

The article "Experimental Study of Multithreading to Improve Memory Hierarchy Performance of Multi-core Processors for Scientific Applications" by Enes Bajrovic and Eduard Mehofer studies the performance characteristics and parallelization strategies for powerful multi-core processors for high-end scientific computing[13]. The article investigates excellent and bad data locality by modifying data access. As well as the impact of multithreading based on the data-parallel programming approach. Data locality and multithreading impacts regarding execution time and bandwidth are measured. The results highlight significant improvements in execution time and bandwidth utilization through increased threading and hardware thread usage. It reveals that while the Block-test and Random-test performance gaps persist despite increased threading, the over-threading strategy successfully mitigates poor core-to-core cache locality and processor stalls, notably improving parallelism. Results show a considerable reduction in performance gaps, with factors dropping from 5.5 to 1.4 on Power6 and 2.2 to 1.3 on T2. This suggests that while over-threading is not a panacea, it is a viable strategy for enhancing multicore processor efficiency. Despite some performance trade-offs, the shift towards high-productivity computing emphasizes the benefits of scalable programming paradigms and reduced software complexity.

The article "Modelling of Parallel Threads Synchronization in Hybrid MPI + Threads Programs" by Andrey V. Tabakov and Alexey A. Paznikov [14] discusses a hybrid multi-threaded MPI program. The standard defines four multithreading types: Single, Funneled, serialized, and multiple, each describing different levels of MPI operation engagements by threads. This paper focuses On the Multiple mode, which necessitates synchronizing communication flows with each process. It introduces a method to synchronize threads using a scalable, thread-safe queue with relaxed semantics, aiming to reduce synchronization overhead in multi-threaded MPI programs. The approach enhances the synchronization of internal program threads across virtual machines.

"Performance evaluation of image smoothing on CPU and GPU using multithreading — An experimental approach in high-performance computing" [15] is an article about the different times and processing techniques used during CPU and GPU execution tasks. It also compares the time and efficiency of the CPU and GPU with various architectures and configurations when applying spatial smoothing filters to images with a fixed spatial resolution. The processing speed increased by 20 to 40 times when the GPU was used instead of the CPU. Furthermore, using filter-specific configurations reduced processing time even more.

Multi-threading image processing in single-core and multi-core CPUs using R language [16] compares a program written in R within a single-thread process and a multi-thread process to check for efficiency. R is a fascinating language for this comparison because it is intended to run on

single core and single thread; however, with the download of specific packages, it can be run with multi-threading, which has been found to make image processing much more efficient. R was also found to be an extremely efficient and applicable language for this experiment due to its “richer toolbox for image analysis and signal processing”(Ramalakshmi and Kampala, 2017). Finally, as predicted, smaller images are processed much faster and faster than large images.

Aljazi Alammari, Mohamed Alkaabi, Mohamed Bubashait, and Abdulla Alqaddumi’s study on Bitmap Image Processing and Edge Detection Using Sobel Algorithm and Multi-Threaded Parallel Techniques[17] describes their work to create a “multi-threaded, parallel, Sobel edge detection algorithm for image processing”. Image processing is a well-known example of the relevance of parallel computing for increased processing efficiency, and edge detection is an example of one parallel computing image processing technique. As its name goes, Edge detection detects differences in image brightness or sharpness in a picture. It refers to variations as “edges.” This helps outline the most prominent (usually in-focus) subjects and/or details of photos. The study uses parallel programming to detect bitmap image “edges.” It was found that both parallelization and multithreading sped up the process of edge detection in the bitmap images.

The combined work of Xiaofeng Liu, Risheng Liu, Ming Zhu, Wei Zhong, Xin Fan, Zhongxuan Luo, and Wanhui Zhang discusses the development of the “Real Time Multi-thread Underwater Images Enhancement System”[18]. Here, multithreading is again applied to image processing (in a non-specific fashion that makes the framework applicable to other algorithms) to make image processing more efficient. Specifically, it is used to enhance underwater images and videos. A particular method developed offers a way to get an optimal thread number that appropriately balances the efficiency of image processing and enhancement quality.

In “A Large-Batch Orthorectification Generation Method Based on Adaptive GPU Thread Parameters and Parallel Calculation”[19], the authors develop a calculation method to determine the adequate performance of GPU thread parameters in the acceleration of orthorectification processing. Orthorectification stretches an image to match map spatial accuracy by considering location, elevation, and sensor information. A process that is already complex and time-consuming increases in difficulty when multiplying the number of images to process (large-batch). An adaptive optimization strategy for large-batch image orthorectification is a product of the study that can be applied to other processes, such as image mosaicing, block adjustment, and dodging uniform colors(2023).

Feature point extraction of high-resolution images based on multi-thread mode [20] is written by Peng Sun, Haimeng Zhao, Yiyuan Sun, Haiju Yang, and Mingchun Wang as a design for a potential solution to the real-time problem of the SIFT algorithm. The processing scheme developed, based on OpenMP software runtime with multi-thread mode, is indeed a feasible design that has been proven to improve the efficiency of the SIFT algorithm. Some weaknesses, however, include issues with the distribution of image feature points and overlapping areas added.

III. METHODOLOGY

This research focuses on improving the efficiency of image processing through multithreading. With traditional single-thread methods, the processing time is prolonged. The drawn-out time is further emphasized when dealing with large datasets. Thus a multithreaded approach is vital to decrease processing time and improve efficiency. This experiment will compare a multithreaded approach with a non-threading version. The research will be conducted in Python utilizing concurrent futures, threading, and multiprocessing modules.

A dataset of 100 images was collected to conduct the study. As shown in the experimental design diagram, an algorithm was created to complete four processes for each image automatically. Four methods were selected based on their frequency and practicality in image processing. The four processes include type conversion, resizing, segmentation, and annotation. Type conversion involves the conversion between image types to change the data type, such as changing an image with RGB to binary or grayscale. Resizing is making an image smaller or larger without cutting anything out. Since resizing alters image dimensions, it affects the file size and quality. Image resizing is typically done to reduce the size of large files to make them easier to share online. Image segmentation involves dividing an image into multiple segments, regions, or objects. Lastly, image annotation is a computer system assigning metadata to images as captions or keywords. This research will analyze metrics like execution time, CPU utilization, and memory usage across MacOS

and Windows operating systems. The expected outcome will be to demonstrate the effectiveness of multithreading in reducing image processing time.

A diagram of the steps mentioned above that are necessary to complete the research is outlined in Fig. 1.

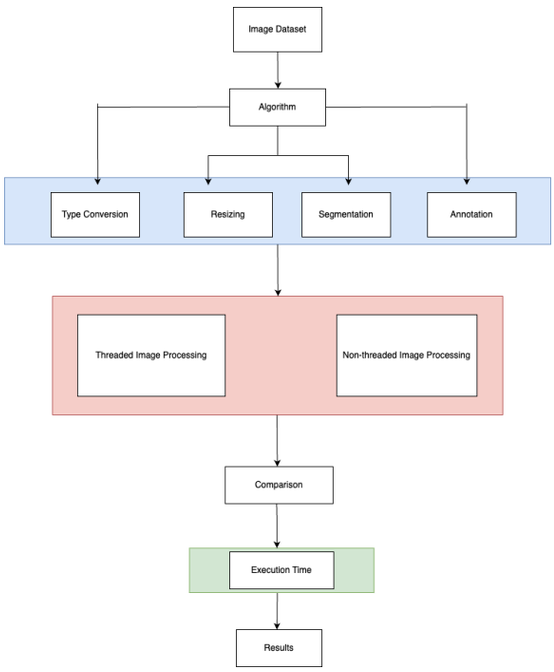


Fig. 1 Diagram of the methodology

A. Dataset Description

Understanding the characteristics and structure of the dataset used in this study is crucial. This dataset is derived from duplicated images from public and private sources. The licensing details of these images are carefully adhered to, ensuring all photos are used within legal boundaries. The dataset comprises one image being duplicated and augmented into a different image multiple times. The data in this study has undergone processing to normalize data inputs for consistency in analysis. This dataset not only serves as a test bed for evaluating the effectiveness of multithreading in image processing but also highlights specific challenges, such as large-scale batch processing. Through this comprehensive dataset, the study aims to demonstrate significant enhancements in processing times and resource utilization to showcase the effectiveness of multithreading in image processing.

B. Implementation

The research can be divided into four steps. The first step is downloading images to create a dataset. The second step is making a single thread script to process all the images in the dataset. The single thread script takes a directory of the photos downloaded and completes four processes for each image: conversion, resizing, segmentation, and annotation. Similarly to the single-thread script, the multithread process script is created and run. Metrics like CPU utilization and execution time are tracked for each script. To conclude the research, a post-processing script is designed to compare the two versions properly.

1) Duplicating images

The first step is downloading images to create a data set. First, data sources are identified. The use of public data sets, web scrapping, and APIs were attempted to be implemented. While trying this method, we ran into limitations that blocked our progress. API limitations, rate limiting, costs, and bot detection were issues experienced. We found that duplicating, data set augmentation, and downloading images were the most efficient way to obtain a diverse dataset. This step was completed by setting a path to an image already downloaded on the user's device. A Python script, “duplicate_image,” was created to duplicate that image 100 times to create a large dataset. This was only for testing as we faced difficulty downloading images from the web. A large dataset of images is necessary to complete the image processing, and most devices do not have a folder to meet that standard. Thus, the impromptu solution of duplicating images was used. Multithreading and multiprocessing techniques for multiple images are used to optimize the downloading process. Effectively, error-handling features are necessary for managing issues like data integrity, connection drops, and corrupt data. The post-downloading process includes data augmentation and the preparation of the dataset for the study.

2) Single-Thread Process Script

The next step in the study was to complete a single-threaded process script. This involves the sequential execution of the four tasks mentioned above: image conversion, resizing, segmentation, and annotation. First, the directory where all the images were downloaded is inputted into the script, and the image processing for each one it comes across is run. For the first method of image conversion, the image. The convert method

translates each RGB image to a pixelated grayscale one. The following image resizing method utilizes the resize method to convert all images to have a width of 500 and a height of 500. The new image is then saved. Thirdly, the image undergoes segmentation with the help of the image method. This step involves creating a threshold implemented by normalizing an image that usually is 0-255 to 0-1. The new image will detect edges and analyze contrast and color differences. The final step is image annotation. The script uses the OpenCV library to draw circles around known shapes. This research mainly focused on the faces of this step. After each image undergoes those four processes, they are resaved in the directory, and the execution time and CPU utilization are tracked for the post-processing steps. This information is saved in an Excel sheet for simple post-processing. The multi-thread process script was created and implemented after the single-thread process script was running. The times were saved to compare the final stages of the research, the post-processing.

3) Multi-Thread Process Script

The multi-thread process aims to spread image processing tasks to run parallel on multiple cores simultaneously and increase processing speed. We plan to take our single thread code and modify it to account for several threads instead of just one. This is a relatively straightforward process.

The single-thread process executes image conversion, resizing, segmentation, and annotation tasks sequentially. To increase efficiency, the computer will automatically run these tasks on several cores for multi-thread. This may mean the program runs all four tasks simultaneously on several regions or pixels of the photo. This could also mean the program runs each task on a different core in a sequential factory-style process. Because the code assigns processes to cores automatically, the computer will decide how it distributes tasks among multiple cores.

Once the multi-thread code is ready, we can process the same images on this code as we processed on the single-thread code to compare similarities and differences in processing speed. Because the code can split the processing tasks onto several cores to complete more at once, the images should process much faster than the single Thread code. This is the hypothesis that we are testing.

4) Analysis

This aspect is designed to conduct a comparative project implementation analysis. After the experiment has been carried out, there would be a need for data collection from the multi-threading and single-threading scripts; these results would include information on the time each script takes and how many resources (memory and CPU utilization) were used up during the execution.

With the data collected from our results, a statistical approach would be used to measure the mean, median, and standard deviation using Python libraries like the Pandas library, commonly used for data manipulation. The results also require visualization so there can be a clear comparison between both threading methods used in the project implementation, which would allow for more straightforward data interpretation.

By conducting a performance analysis, efficiency improvements can be evaluated. Data was collected from the reduction in execution using multi-threading and single-threading and a study of CPU resource utilization.

IV. EXPERIMENT

To gain access to an extensive dataset, the initial approach was to develop and utilize a web scrapping tool to download an array of images from Google Images. This would have involved importing various libraries designed for web interaction and HTML parsing. It would act as a user agent, mimic browser requests, and initiate a directory to save images. This approach wasn’t followed as there were issues with API access. The approach used in carrying out the experiment involved downloading an image and duplicating it several times before it could be processed.

Importing Python libraries like ‘shutil’ and ‘os’ are used to carry out file operations. A script was developed to define a path to the source image for duplication and the destination folder to store all duplicated images. The library ‘os.makedir’ checks if the output directory exists before sorting the duplicated images. A loop is created to copy the source image several times, and each time it saves it under a filename in the created output directory, this operation also carries out error handling and ensures the image is duplicated the correct amount of times.

After the images have been duplicated, two methods are used in image processing; single threading and multithreading. Python libraries ‘PIL’, an imaging library, ‘cv2’ are used for advanced image manipulation, ‘concurrent.futures’ is used to enhance processing speed through multithreading, and ‘psutil’ is used to monitor system performance during processing. These libraries were used to access various functions, such as grayscale and binary conversion to adjust the images' color format, resizing to standardize pictures to a set dimension and preserve the aspect ratio, segmentation achieved by utilizing k-means clusters to segment the images in color-based clusters, and annotation for drawing shapes on images.

The script uses a multithreading and single threading process to handle image processing tasks, including variables like time spent and CPU resource utilization, which are measured. By connecting the output from the image duplication process, these images can get annotated and segmented.

V. RESULTS

The experiment was carried out on various devices that contained different processing cores, which would also be a factor in the difference in the result. The variables measured across the devices were the time taken to complete each process and the resources used up by the CPU. The devices used are a Thinkpad with an Intel Core, an Apple MacBook Pro containing an Intel Core, an Asus using an AMD Ryzen core, and an HP 360x Envy using an AMD Rzyen core.

Table 2 below summarizes the computer specifications used for this experiment.

Computer Specifications
AMD Ryzen 7 4800H
AMD Ryzen 7 5000 Series
Macbook Pro (13 in, M2, 2022)
ThinkPad X1 Yoga Gen 6

Across all devices, switching from a single-threaded process to a multi-threaded process significantly reduced processing time from several minutes to less than a second. These results show multithreading allows a program to handle several operations simultaneously, speeding up a large dataset.

To demonstrate the image processing completed for this experiment, below are the before and after of an image when the Python script is run. Figure 2 represents the before image, and Figure 3 illustrates the after picture.

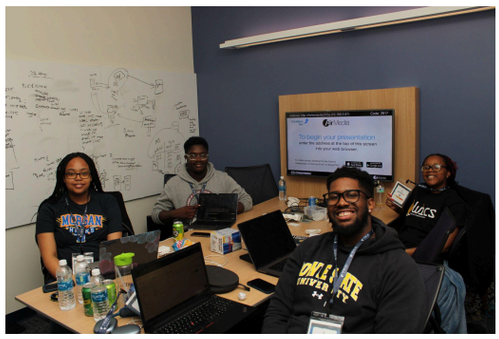


Fig. 2 Before Image



Fig. 3 After Image

Below Table 1 summarizes the execution time and CPU utilization for running the single thread and multithread scripts on the four computers.

TABLE I
Comparison of Multithreading and Single Threading

	Single Thread		Multi Thread	
	Time	CPU	Time	CPU
ThinkPad	450.46 secs	41.3	0.02 secs	12.5
MacBook Pro	217.96 secs	62.9	0.01 secs	33.3
Asus	371.89 secs	6.7	0.01 secs	22.2
HP 360x	352.85 secs	7.1	0.08 secs	2.2

Envy				

The MacBook Pro shows a high CPU utilization in single threading and multithreading processes, indicating a wide utilization of its CPU resources, the HP and ThinkPad show lower CPU usage in the multithreading process, which may be an indication of underutilization or efficient CPU management depending on the task demands. The variation in the results shows the importance of tailoring software to use each system's hardware effectively; this optimizes performance and resource utilization.

VI. CONCLUSION

The experiment was put in motion by evaluating the effectiveness of multithreading in enhancing efficiency and the speed of image processing, particularly with an extensive image dataset. The results demonstrate multithreading's effectiveness over traditional single-treading processing methods by reducing the time of execution and resource utilization significantly across multiple operating systems.

The experimental design and implementation of the paper utilize systematically analyzing and comparing the performance of a multithreading process against a non-threading process. The implementation also uses an extensive dataset containing 100 images.

By discussing the consistent gains in CPU utilization and memory management, the paper sheds some light on the implications of multithreading in various computing environments. These findings highlight how multithreading distributes processing tasks across multiple cores, ensuring more efficient data handling and processing.

This research would contribute significantly to computational efficiency by offering an empirical basis for adopting multithreading techniques in data-intensive applications and processes. This provides an avenue for future studies exploring the integration of advanced threading methods and how they can be applied in real-world scenarios where processing efficiency is crucial. Multithreading in image processing enhances performance and efficiency. It aligns with current trends in software development and the need for more robust and efficient processing techniques in the era of big data.

REFERENCES

[1] S. K. Shammi, S. Sultana, M. S. Islam and A. Chakrabarty, "Low Latency Image Processing of Transportation System Using Parallel Processing co-incident Multithreading (PPcM)," 2018 Joint 7th International Conference on Informatics, Electronics & Vision (ICIEV) and 2018 2nd International Conference on Imaging, Vision & Pattern Recognition (icIVPR), Kitakyushu, Japan, 2018, pp. 363-368, doi: 10.1109/ICIEV.2018.8640957.

[2] Q. W. Samyan, W. Sahar, W. Talha, M. Aslam, and A. M. Martinez-Enriquez, "Real-Time Digital Image Processing Using Point Operations in Multithreaded Systems," 2015 Fourteenth Mexican International Conference on Artificial Intelligence (MICAI), Cuernavaca, Mexico, 2015, pp. 52-57, doi: 10.1109/MICAI.2015.15.

[3] Y. Liu, B. Chen, H. Yu, Y. Zhao, Z. Huang, and Y. Fang, "Applying GPU and POSIX thread technologies in massive remote sensing image data processing," 2011 19th International Conference on Geoinformatics, Shanghai, China, 2011, pp. 1-6, doi: 10.1109/GeoInformatics.2011.5980671.

[4]O. Nakov, E. Mihaylova, M. Lazarova and V. Mladenov, "Parallel Image Stitching Based on Multithreaded Processing on GPU," 2018 International Conference on Intelligent and Innovative Computing Applications (ICONIC), Mon Tresor, Mauritius, 2018, pp. 1-5, doi: 10.1109/ICONIC.2018.8601253.

[5] S. Bose, A. Mukherjee, Madhulika, S. Chakraborty, S. Samanta, and N. Dey, "Parallel image segmentation usingeeading and k-means algorithm," 2013 IEEE International Conference on Computational Intelligence and Computing Research, Enathi, India, 2013, pp. 1-5, doi: 10.1109/ICCIC.2013.6724171

[6] L. S. G. Kovasznay and H. M. Joseph, "Image Processing," in Proceedings of the IRE, vol. 43, no. 5, pp. 560-570, May 1955, doi: 10.1109/JRPROC.1955.278100

[7] Malakhov, Anton. (2016). Composable Multi-Threading for Python Libraries. 10.25080/Majora-629e541a-002.

[8] Basermann, Achim & Röhrig-Zöllner, Melven & Illmer, Joachim. (2015). Performance and productivity of parallel python programming: a study with a CFD test case. 1-10. 10.1145/2835857.2835859.

[9] Pang, Yingxue & Lin, Jianxin & Qin, Tao & Chen, Zhibo. (2021). Image-to-Image Translation: Methods and Applications.

[10] Y.-S. Wang, C.-L. Tai, O. Sorkine, and T.-Y. Lee, “Optimized scale-and-stretch for image resizing,” *ACM Transactions on Graphics*, vol. 27, no. 5, pp. 1–8, Dec. 2008, doi: https://doi.org/10.1145/1409060.1409071.

[11]J. Bo, "Improved Parallel Downloading Algorithm for Large File Distribution," 2012 International Conference on Computer Distributed Control and Intelligent Environmental Monitoring, Zhangjiajie, China, 2012, pp. 373-376, doi: 10.1109/CDCIEM.2012.95.

[12]Y. Zhang, "The studies and implementation for conversion of image file format," 2015 10th International Conference on Computer Science & Education (ICCSE), Cambridge, UK, 2015, pp. 190-193, doi: 10.1109/ICCSE.2015.7250241

[13]E. Bajrovic and E. Mehofer, "Experimental Study of Multithreading to Improve Memory Hierarchy Performance of Multi-core Processors for Scientific Applications," 2009 International Conference on Complex, Intelligent and Software Intensive Systems, Fukuoka, Japan, 2009, pp. 645-650, doi: 10.1109/CISIS.2009.110.

[14]A. V. Tabakov and A. A. Paznikov, "Modelling of Parallel Threads Synchronization in Hybrid MPI + Threads Programs," 2019 XXII International Conference on Soft Computing and Measurements (SCM)), St. Petersburg, Russia, 2019, pp. 197-199, doi: 10.1109/SCM.2019.8903806.

[15]G. Sethumadhavan, S. A. Narayanasamy, and A. Gopalakrishnan, "Performance evaluation of image smoothing on CPU and GPU using multithreading — An experimental approach in high-performance computing," 2016 IEEE International Conference on Computational Intelligence and Computing Research (ICCIC), Chennai, India, 2016, pp. 1-5, doi: 10.1109/ICCIC.2016.7919680.

[16]E. Ramalakshmi and N. Kampala, "Multi-threading image processing in single-core and multi-core CPU using R language,"

2017 Second International Conference on Electrical, Computer and Communication Technologies (ICECCT), Coimbatore, India, 2017, pp. 1-5, doi: 10.1109/ICECCT.2017.8117873.
[17]A. Alammari, M. Alkaabi, M. Bubashait and A. Alqaddumi, "Bitmap Image Processing and Edge Detection Using Sobel Algorithm and Multi-Threaded Parallel Techniques," 2021 International Conference on Data Analytics for Business and Industry (ICDABI), Sakheer, Bahrain, 2021, pp. 493-499, doi: 10.1109/ICDABI53623.2021.9655978.
[18]X. Liu et al., "A Real-Time Multi-thread Underwater Images Enhancement System," 2020 International Conference on System Science and Engineering (ICSSE), Kagawa, Japan, 2020, pp. 1-5, doi: 10.1109/ICSSE50014.2020.9219259.
[19]R. Zhou et al., "A Large-Batch Orthorectification Generation Method Based on Adaptive GPU Thread Parameters and Parallel Calculation," in IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, vol. 16, pp. 4638-4648, 2023, doi: 10.1109/JSTARS.2023.3276219.
[20]P. Sun, H. Zhao, Y. Sun, H. Yang and M. Wang, "Feature point extraction of high-resolution image based on multi-thread mode," 2020 IEEE 2nd International Conference on Civil Aviation Safety and Information Technology (ICCASIT, Weihai, China, 2020, pp. 836-839, doi: 10.1109/ICCASIT50869.2020.9368557.

APPENDIX

If you would like to see the code utilized for this experiment, click the link below:
https://github.com/DaBombShan11/COSC-414---Group-9